pic-sim Manual

pic-sim and basil-sim 0.2.3 · PIC18 and PIC16 simulator

2026-11-07

pic-sim Manual

Contents

About this manual 4

1. Overview 5
- What it is 5
- What "cycle exact" means 5
- What it is not 5

2. Installing and starting 6
- Build 6
- Where the device descriptions are found 6
- Starting 6

3. Preparing a program 7

4. Supported devices 8

5. The graphical simulator: pic-sim 9
- The window 9
- Loading a program 9
- The toolbar 10
- Run speed 10
- The Program pane 11
- The CPU panel 11
- The data tabs 12
- The board strip 13
- Typical sessions 13

6. The command-line simulator: basil-sim 14
- Synopsis 14
- Options 14
- Exit status 15
- The report 15
- Examples 16

7. The stimulus file 18
- Pins: `[[pin]]` and `[[port]]` 18
- Serial input: `[[uart]]` 18
- ADC input: `[[adc]]` 19
- Analogue pin voltage: `[[analog]]` 19
- I2C client: `[[i2c]]` 19
- SPI client: `[[spi]]` 19
- Reset pin: `[[mclr]]` 20
- Clock for the display: `fosc_mhz` 20
- Board parts: `[[led]]` and `[[oled]]` 20
- What each family accepts 20
- Examples from the repository 20

8. The simulated board: LEDs and OLED 22
- LEDs 22
- SSD1306 OLED 22

9. What is simulated 24
- Time 24

pic-sim Manual

Cores 24
Peripherals 24
Configuration words 25
Deliberate differences from MDB 26

10. Troubleshooting 27

11. Appendix A: quick reference 28
pic-sim controls 28
basil-sim options 28
Stimulus sections 28
Pin names 28
Run speeds 28
Exit status of basil-sim 29

12. Appendix B: example session 30

About this manual

Revision for **pic-sim 0.2.3** and **basil-sim 0.2.3** (one workspace version). Simulated families: **PIC18** (PIC18F27Q10 and the PIC18F04/05/06/14/15/16Q41) and **PIC16 enhanced mid-range** (PIC16F131xx, 152xx, 171xx, 180x6 and 183xx).

This manual describes what the simulator does today. It is organised in the order you meet the tools: what the simulator is, how to build and start it, how to prepare a program for it, the graphical front end `pic-sim`, the command-line front end `basil-sim`, the stimulus file that describes the outside world, the simulated board (LEDs and OLED), and finally what is and is not modelled. Anything not listed here is not supported.

Status: the simulator is **cycle exact against the data sheets and, where a Microchip MDB (MPLAB X simulator) comparison was possible, against MDB**. It has **not been compared with silicon**. Where the data sheet is silent the simulator makes an assumption; each one is numbered `[A#]` and written down with its source in `docs/FACTS-SIM.md`. The few places where the simulator deliberately disagrees with MDB are numbered `D-SIM-#` in the same file.

Related documents: `docs/FACTS-SIM.md` (every timing rule with its data sheet page), `docs/SIMULATOR-PLAN.md` (architecture and milestones), `docs/DEVICE-FORMAT.md` (the device description files), `docs/REVIEW-FINDINGS.md`. The compiler that produces the programs is **basil**; its manual is `Basil_Manual.pdf` in the Basic_Compiler project.

1. Overview

What it is

The workspace contains one simulation library and two programs that use it:

Program	Kind	Use it to
`pic-sim`	Window (egui)	Load a program, run it, step it by instruction or by BASIC source line, set breakpoints, watch registers, memory and variables, wire LEDs and an OLED display to pins, time code with a stopwatch
`basil-sim`	Command line	Run a program for a number of cycles or until it halts, then print variables, serial/SPI/I2C traffic, pin changes, LED history and the OLED picture; drive scripted tests and benchmarks

Both load the `.hex` file that the **basil** compiler writes, and both read the same **stimulus** file (a TOML file that says what the outside world does to the pins, serial receiver, ADC and so on). Both use the same simulated parts, so a result seen in one is the result in the other.

What "cycle exact" means

The simulator executes the instructions of the part exactly as the hex file encodes them and counts **instruction cycles** (one cycle is four oscillator clocks, `Fosc / 4`). The peripherals are modelled to the cycle as well: a timer prescaler, the delay before an interrupt is taken, the time a UART needs per character or an ADC conversion needs per sample. The cycle counter in the window and in `basil-sim`'s `stop:` line is therefore the number of cycles the real part would take, and dividing by `Fosc / 4` gives real time.

Because time is simulated, it is independent of how fast the host runs the simulation. A program that waits a simulated second can finish in milliseconds of host time (speed *Max*), or be slowed to real time or a fraction of it (section 5, *Run speed*).

What it is not

- It is not a hardware debugger. You cannot change a register or memory cell from the window; you change the program or the stimulus and run again.
- Only peripherals that are modelled behave. A register belonging to a peripheral without a model is plain storage, and both programs tell you which such registers the program touched (section 9).
- Analogue behaviour is not simulated. The ADC converts a code that you supply; pull-ups, drive strength and input thresholds are not modelled.

2. Installing and starting

Build

You need a Rust toolchain that supports the 2024 edition (Rust 1.85 or newer). From the top of the workspace:

```
cargo build --release
```

This produces two programs in `target/release/`:

File	Program
`pic-sim`	the graphical simulator
`basil-sim`	the command-line simulator

`cargo test --release` runs the test suite (core, peripherals, board, debug information). `pic-sim --version` and `basil-sim --version` print the version, which is also shown in the window title.

On Linux the window needs the usual graphics libraries for an `egui` program (X11 or Wayland and OpenGL); file dialogs are provided by the `rfd` crate.

Where the device descriptions are found

The simulator needs one description file per part, `devices/<part>.toml` (section 4). The folder is found at **build time**: the programs look for `devices/` at `<workspace>/devices`, the path they were built from. Consequences:

- Run the programs from where `cargo` built them, or keep the workspace where it was when you built it. If you move the workspace, rebuild.
- `basil-sim` can be pointed elsewhere with `--devices DIR`. `pic-sim` has no such option; if its device box is empty, the `devices/` folder could not be found.

Starting

```
pic-sim [prog.hex | prog.elf] [--device PART]
basil-sim prog.hex|prog.elf [--device PART] [options]
```

With no argument `pic-sim` opens empty; type a path or use *Browse hex/elf…*. `basil-sim` needs a program. The default part is `18F27Q10` for `basil-sim`; `pic-sim` starts with the first PIC18 part in its list. Because a program built with `basil -g` records the part it was built for, you normally do not need `--device` in the window (section 5).

3. Preparing a program

The simulator runs what **basil** produces. Compile with:

```
basil blink.bas            # writes blink.hex, blink.S, blink.map
basil -g blink.bas         # also writes blink.elf: lines, variables, routines
basil --dual blink.bas     # release build plus an unoptimised debug build (blink_debug.*)
```

Which files matter:

File	Needed?	What the simulator takes from it
`<name>.hex`	**Yes**	The program image, including configuration words and EEPROM contents
`<name>.elf`	Recommended	Written by `basil -g`. Gives routines, the `__halt` address, global variables with their types, each routine's parameters and locals, and the address-to-BASIC-line map. It also names the part it was built for
`<name>.S`	Optional	The assembly listing. Gives code labels and variable addresses for `--dump`, `--bp`, `--until` and the Watch tab. If both an ELF and a listing exist, the listing's symbols win

Rules for finding the files:

- You may give either the `.hex` or the `.elf`. Given an `.elf`, the simulator loads the `.hex` of the same name beside it, or in a `dist/` folder next to it.
- The ELF is looked for beside the hex (`prog.elf`), or one folder up (`../prog.elf`), because basil writes the hex into `dist/` in a project build.
- The listing is looked for as `prog.S` beside the hex (the GUI also tries `prog_basil.S`).

Debug builds. Without `-g` there is only the hex (and the listing). You can still run, step instructions, set breakpoints on addresses and watch SFRs. The Source view, line stepping, the Variables tab and routine names in the call stack all need the ELF. In VS Code use *Build Debug* (Ctrl+F7); the plain build (F7) has no line information. `--keep-vars` makes basil keep stores to variables nothing reads, which scripted tests need so that a variable can be dumped at the end.

The halt address. basil places a label `__halt` where the program ends. `basil-sim` runs *until* that address by default, and the window stops there and reports it. A program that never ends (a `While True` main loop) never reaches `__halt`; it runs until `--max-cycles` or until you press Pause.

4. Supported devices

The parts the simulator can load are the files in `devices/`:

Family	Parts	Core	Notes
PIC18-Q10	18F27Q10	PIC18	The 28-pin part; the 40-pin variant is not separately described
PIC18-Q41	18F04Q41, 18F05Q41, 18F06Q41, 18F14Q41, 18F15Q41, 18F16Q41	PIC18	Interrupts with `MVECEN` off (see section 9)
PIC16F131xx	16F13113, 13114, 13115, 13123, 13124, 13125, 13143, 13144, 13145	PIC16 enhanced	
PIC16F152xx	16F15213, 15214, 15223, 15224, 15243, 15244	PIC16 enhanced	
PIC16F171xx	16F17126, 16F17146	PIC16 enhanced	
PIC16F180x6	16F18056, 16F18076	PIC16 enhanced	
PIC16F183xx	16F18313, 18323, 18324, 18325, 18326, 18344, 18345, 18346 and the **LF** variants 16LF18313 … 16LF18346	PIC16 enhanced (5-bit BSR)	

The window's device box lists only the parts whose file name starts `18F` or `16F`, so the `16LF…` parts can be used with `basil-sim --device 16LF18346` but are not offered by `pic-sim`'s box.

Device files are generated from Microchip's device packs by `tools/devgen` and cross-checked against the data sheets; they are never edited by hand (`docs/DEVICE-FORMAT.md`). Each carries the memory map, the special function registers with reset values and read/write masks, the configuration words, the interrupt vectors and the pin list.

5. The graphical simulator: pic-sim

The window

```
+-------------------------------------------------------------+
| TOOLBAR: device | hex/elf path | stimulus | Browse | Stim |
|   Load | Run Pause Step ... Reset | Speed | Fosc MHz         |
| status line                                                 |
+----------+---------------------------+----------------------+
| CPU      | Program                   | Memory  SFRs  Watch  |
|  regs    |  Source | Disassembly     |  Board  Variables    |
|  Stop-   |  [x] follow PC            |                      |
|  watch   |  [x] stay in my code      |  (the selected tab)  |
|  Call    |  goto [      ] Go         |                      |
|  stack   |  > 0042  ...              |                      |
|  Notes   |                           |                      |
+----------+---------------------------+----------------------+
| BOARD STRIP: LEDs and OLED (shown only when wired)          |
+-------------------------------------------------------------+
```

The window title shows `pic-sim <version>`. The window is divided into the **toolbar** and **status line** at the top, the **CPU panel** at the left, the **Program pane** in the middle, the **data tabs** at the right, and the **board strip** along the bottom when LEDs or an OLED are wired.

Loading a program

1. Pick the part in the device box, or leave it: when the ELF names the part it was built for, that part is selected automatically and the status line says `device set to … from the ELF`. The wrong part would have the wrong registers and pins, so the ELF wins over the box. (This works only for parts the box lists, so not for the `16LF…` parts: choose those by hand.)
2. Enter a path in the first text field, or press **Browse hex/elf...**, or drag a file onto the window. A `.hex` or `.elf` dropped on the window loads at once (ignored while the program is running). A `.toml` dropped on the window becomes the stimulus file.
3. Optionally enter a stimulus file in the second field, or press **Stim...**. A stimulus file is a TOML file that **you write** (section 7) to feed the program serial input, ADC values, pin levels and so on; none is supplied with your program, and you can leave the field empty to run without one. The **Stim...** dialog opens in the folder of the stimulus file already named, else in the program's folder, else in the repository's `tests/sim` folder, which holds working examples. The small **x** beside the field clears it. **Browse hex/elf...** likewise opens in the program's folder. The stimulus is read when you press **Load**, so choosing one after loading needs another **Load**.
4. Press **Load**.

The status line then reads `loaded <path> (N symbols, M variables)` followed by a note when something is missing: `no .elf beside the hex: no source view or variables`, or `ELF has no BASIC lines (build with basil -g)`. An error (for example a bad stimulus file) is shown there and nothing is loaded.

If the ELF holds variables, the **Variables** tab is selected automatically.

Pressing **Load** again always starts afresh: the program is reloaded, the cycle counter is 0, the stopwatch is cleared, any step in progress is cancelled and the breakpoints are cleared. The wired LEDs and OLED are kept (section 8).

The toolbar

Control	Action
Run	Run the program. Enabled when a program is loaded and nothing is running or stepping. The program runs until it halts, reaches a breakpoint, goes to Sleep with nothing able to wake it, or you press Pause
Pause	Stop a run or cancel a step in progress. Enabled while running or stepping
Step	Execute one instruction
Step over	Like Step, but a call (PIC18: `CALL`, `RCALL`; PIC16: `CALL`, `CALLW`) runs until it returns to the following instruction. Everything else is one step. A long step over runs in the background, keeps the window alive, and can be cancelled with Pause
Line	Step to the next BASIC source line, going into called routines. Needs the ELF with BASIC lines
Line over	Step to the next BASIC line, running calls and interrupts to completion
Out	Run until the current routine returns
Reset	Restart the part from its reset vector with cycle counter 0 and the stimulus replayed from the start. The wiring of LEDs and OLED stays (the OLED starts blank, as after power-up). Clears the stopwatch
Speed	Run speed (below)
Fosc MHz	The oscillator frequency used to turn cycles into time (0.03 to 64). It does not change the program's own idea of its clock; it is the clock the *display* assumes. PIC16 parts are limited to 32 MHz when loaded

While a step is in progress, **Run**, **Step**, **Step over**, **Line**, **Line over** and **Out** are disabled and only **Pause** is available; the status line shows `stepping... (Pause cancels)`. A source-line step (Line, Line over, Out) also stops early at a breakpoint, at `__halt`, or after about a million instructions.

When a run or step ends the status line shows the reason after the loading message, for example `loaded … — stopped: Halt`. The reasons are:

Shown	Meaning
`Halt`	The program counter reached `__halt` (or the target of a Step over)
`Breakpoint(addr)`	A breakpoint address was reached
`Sleep`	`SLEEP` was executed and nothing that is enabled can wake the part
`Cycles`	The cycle budget ran out (`basil-sim` only)

Run speed

The **Speed** selector sets how fast simulated time passes against real time:

Setting	Meaning
Max	As fast as the host allows. A slider *cycles/frame (10^x)* appears and sets how much is simulated per screen update (10^0 … 10^7; the default is 10^6)
1x (real time)	The simulated part runs as fast as the real one at the chosen Fosc

Setting	Meaning
1/2, 1/4, 1/10, 1/100, 1/1000	A fraction of real time

Use *1x* or a fraction to watch an LED blink or a display scroll at a speed you can see, and Max to get through initialisation. The Fosc box sets the clock for this calculation: one instruction cycle is `Fosc / 4`, so at 64 MHz real time is 16 million cycles a second. Changing the speed or Fosc while running takes effect immediately without a jump. If the host stalls, the simulator never catches up by more than 0.25 s of real time, so the part does not race through a burst.

While running, the status line shows the achieved rate, for example `running at 100.0% of real time (16.00 Mcycles/s)`. It also reminds you that a wait on a UART or timer flag shows as one source line for a long time — that is the program waiting, not the simulator.

The Program pane

The pane has two views, chosen with the **Source** / **Disassembly** buttons (the buttons appear only when the ELF supplies BASIC lines).

Source view. Shows the BASIC file with line numbers. The line the program counter is on is highlighted green, a breakpoint is a red ●. With **follow PC** on, the view scrolls to keep the current line centred. When the program counter is in generated code that has no BASIC line (start-up code, the compiler's helper routines), the view stays on the last source line and says so. Library routines written in BASIC (`i2c.bas`, for example) show their own file. With **stay in my code** on, a library routine that your code called is shown as the line of your program that made the call, so a `DelaySeconds` appears as one line instead of a jump into the library.

Click a source line to toggle a breakpoint on it; the breakpoint goes on the lowest address of the line. A comment or blank line has no code and cannot take a breakpoint. The compiler's optimiser can merge lines, so some lines have no code.

Disassembly view. Shows address, instruction word and mnemonic. The PIC18 listing is in byte addresses; the PIC16 listing is in **word** addresses (the PIC16 program counter counts words). Labels from the listing or ELF appear in blue above their instruction. The current instruction is highlighted; a breakpoint is a red ●. Click an instruction to toggle a breakpoint at it. The text field *goto 0x…* with **Go** moves the listing to an address (PIC16: a word address, 15 bits) and turns *follow PC* off.

If the build has no BASIC line information, an orange message at the top of the pane says so and tells you how to build with it.

The CPU panel

The left panel shows the core registers and the cycle counts.

PIC18: PC, WREG, BSR, STATUS (with N, OV, Z, DC, C shown as letters when set), FSR0 to FSR2, PROD, TBLPTR, TABLAT, STKPTR.

PIC16: PC (word address), WREG, BSR, STATUS (Z, DC, C), PCLATH, INTCON, FSR0, FSR1.

Both show **Cycles** (instruction cycles since reset), **Instr** (instructions executed; some take two or more cycles) and **Time** (cycles at the Fosc box's clock, in milliseconds).

Stopwatch

The stopwatch measures the interval between two moments of the *simulated* run, in cycles and in microseconds at the Fosc box. Because it reads the simulated cycle counter it is exact whatever the run speed.

Button	Action
Start	Start counting from zero, or resume after Stop. The time spent stopped is left out
Stop	Freeze the reading
Lap	Record the elapsed time (enabled while counting). Each lap is listed as `Lap n: <total> us (+<since previous lap>)`
Clear	Zero the stopwatch and the laps

A typical use: set a breakpoint at the start and end of a routine, press Start when the first is hit, Run to the second, press Stop, and read the time the routine took. A Reset or a Load clears the stopwatch because the counter it was measuring has restarted.

Call stack

With the ELF, the panel is headed **Call stack**: the first line (`>`) is the routine and BASIC line the program counter is in, followed by the routine and line of each pending return, innermost first (up to 12 on a PIC18, 16 on a PIC16). Without the ELF it is headed **Return stack** and lists the return addresses (on a PIC16 with the nearest label and offset).

Notes

When the simulator meets something it does not model or that is suspect, it adds a note here, for example `MVECEN = 1 (vector table) is not simulated`, an OLED command with no effect, or `unmodelled SFRs touched: …`. Treat notes as the simulator telling you where the result is not backed by a model.

The data tabs

Memory

A hex dump of data memory, 16 bytes a row with an ASCII column. On a PIC16 an address that has no implemented bit shows `--`. The view is read-only.

SFRs

A list of the special function registers with address, hex and binary value, in alphabetical order. The *filter* box matches part of a name (case-insensitive). Reading is without side effects: viewing a register does not clear a flag or pop a receive buffer.

Watch

Enter `symbol` or `symbol:bytes` and press **Add** to watch a value; `bytes` is 1 to 8 (default 1). The symbol may be a code or data label from the listing, a global variable from the ELF, or an SFR name (`TMR0L`, `PIR0`). Each entry shows address and value as unsigned, signed and hex, for example `T_R1[2] @0123 = 1000 (1000) 0x3E8`. Multi-byte values are little-endian. A name that is not found shows `unknown`. Press `x` to remove an entry.

Board

Wire LEDs and an OLED display to pins (section 8). Available even before a program is loaded.

Variables

The globals of the program and the parameters and locals of the routine the program counter is currently in, with type, data address and value:

- `Integer` is shown signed, `Char` / `String` as text, `Float` as a binary32 number, arrays element by element.
- A parameter is tagged `(arg)`.
- The heading says `in <routine>` or `in start-up code (globals only)`.
- A value that changed since the last Step or Run began is shown in **orange**. The comparison is against a snapshot taken when you pressed Run or a step button, so after a Run to a breakpoint the orange variables are those that the run changed.

The values are live while running. Without the ELF the tab only says that variable information needs `basil -g` and the ELF beside the hex.

The board strip

When LEDs or an OLED are wired, a strip along the bottom shows them. An LED is a disc whose brightness follows the fraction of time the pin was at the active level over the last window of time, so a PWM-dimmed or multiplexed LED shows an intermediate brightness. An OLED is drawn at three times its size in light blue on black; if the OLED is not answering (wrong pins, wrong address, missing PPS set-up) its caption says `NOT WIRED:` and why.

Typical sessions

Find why a variable has the wrong value. Build with `basil -g`, load, click the source line where the variable is set, Run to it, then Line / Line over and watch the Variables tab for orange entries.

Time a routine. Breakpoints at its first and last line, Stopwatch as above.

Check an output waveform. Wire an LED to the pin, run at 1/10 or 1/100 speed and watch it, or use `basil-sim` and read the change list (section 6).

Test serial input. Write a stimulus file with a `[[uart]]` section, load it with the program, and Run; the characters arrive at the receiver at the cycle you chose. Watch the receive variables in the Variables tab.

6. The command-line simulator: basil-sim

Synopsis

```
basil-sim prog.hex|prog.elf [--device PART] [--devices DIR]
          [--asm prog.S] [--until SYMBOL] [--max-cycles N]
          [--stim FILE] [--dump NAME[:BYTES],…]
          [--bp SYMBOL,…] [--bp-max N]
          [--trace FROM-TO] [--config]
          [--speed max|RATIO] [--fosc MHZ]
basil-sim --version
```

The program runs from reset until it stops, then `basil-sim` prints a report and exits.

PIC16 parts report less. The PIC16 path prints the stop line, `--bp` lines (without the timer values), `--dump` values, `tx` text and the notes. It does not print the `spi`, `i2c`, `pins`, `led` and `oled` lines, and `--config` is refused (`--config is not supported for PIC16 parts yet`). For LEDs and an OLED on a PIC16 part use `pic-sim`, which draws them. The descriptions below say *(PIC18 only)* where this applies.

Options

Option	Default	Meaning
`prog.hex` / `prog.elf`	required	The program. Given an `.elf`, the hex beside it (or in `dist/`) is loaded
`--device PART`	`18F27Q10`	The part, as the name of a file in `devices/` without `.toml` (`18F16Q41`, `16F17146`, `16LF18346`). The part is **not** taken from the ELF here, so give the right one
`--devices DIR`	`<workspace>/devices`	Folder holding the device files
`--asm FILE`	`<hex>.S`	The assembly listing to take symbols from
`--until SYMBOL`	`__halt`	Stop when the program counter reaches this symbol. The symbol must exist (in the listing or ELF) or the run does not start
`--max-cycles N`	2 000 000 000	Stop after this many instruction cycles even if the symbol was not reached
`--stim FILE`	none	Stimulus file (section 7)
`--dump LIST`	none	After the run, print these symbols or SFRs; `NAME` is one byte, `NAME:N` is *N* bytes read little-endian
`--bp LIST`	none	Comma-separated code symbols. Each time the program reaches one, a line is printed and the run continues
`--bp-max N`	20	Stop after this many breakpoint hits
`--trace FROM-TO`	off	Run silently to cycle FROM, then print every instruction until cycle TO, then exit
`--config`	off	Print the configuration words and the oscillator and watchdog registers they set (PIC18 only)

Option	Default	Meaning
`--speed max|R`	`max`	Limit the run to a fraction of real time: `1` is real time, `0.5`, `1/10`, `0.001` …
`--fosc MHZ`	64	The clock `--speed` uses (0 < MHZ ≤ 64)
`--version`, `-V`		Print `basil-sim <version>` and exit

An unknown option, a missing value, an unreadable file, an unknown device, a bad stimulus file or an unknown symbol stops with `basil-sim: <message>` on standard error and exit status 2.

Exit status

Status	Meaning
0	The program reached `--until` (by default `__halt`)
1	The run stopped for another reason: cycle limit, breakpoint limit, Sleep
2	An error (see above); nothing was simulated

This makes `basil-sim` usable in scripts: a test program that ends at `__halt` returns 0, one that hangs hits `--max-cycles` and returns 1.

The report

After the run, `basil-sim` prints these lines, each only when it has something to show.

Stop line (always).

```
stop: Halt  cycles: 30414  instructions: 20118
```

The reason is `Halt`, `Cycles`, `Sleep` or `Breakpoint(address)`, followed by the instruction cycles and instructions executed.

Breakpoint lines. One for each hit of a `--bp` symbol, before the stop line:

```
bp 1 T_LOOP cycles 1204 tos 0x0018 TMR0L 17 TMR0H 0
```

On a PIC18 the line also shows `TMR0L`/`TMR0H`; `tos` is the top of the return stack.

Dumps. For `--dump T_R1:2,TMR0L`:

```
T_R1 = 1000  (E8 03)
TMR0L = 17  (11)
```

The decimal value is the little-endian combination of the bytes, followed by the bytes in memory order.

Serial. `tx1: …` and `tx2: …` show what the EUSART/UART transmitted, as a string with control characters escaped (`\r\n`).

SPI (PIC18 only). `spi1: 9F 00 00` lists the bytes the program sent through SPI 1, in hex.

I2C (PIC18 only). `i2c1: S 6FW! P` is the bus activity of the hardware I2C host: `S` start, `Sr` repeated start, `P` stop, an address as `<7-bit hex>` followed by `R` or `W`, data bytes in hex, and a `!` after an address or written byte that nobody acknowledged, or after a byte the host answered with NACK (the last byte of a read). The example is a write to address 6F that no device answered. A complete transaction with a client at 50 looks like `S 50W 10 Sr 50R CA FE! P`.

Pin log (PIC18 only). `pins: RC3=1@4 RC3=0@96 …` lists changes of pins that a peripheral drives (PPS outputs: PWM, serial TX, SPI, CWG …), as `R<port><bit>=<level>@<time>`. The time is in **oscillator ticks**, four per instruction cycle.

LEDs (PIC18 only). One line for each LED in the stimulus file:

```
led heartbeat (RB0, active high): ON, on 11999995 of 20000001 cycles, 3 changes: 1@2
0@8000004 1@16000008
led error (RB1, active low): off, on 8000006 of 20000001 cycles, 4 changes: 1@1 0@3
1@8000005 0@16000009
```

It gives the final state, the cycles lit out of the total, the number of changes and the first 40 changes as `level@cycle`.

OLED (PIC18 only). A line `oled1 SSD1306 0x3c on SCL RC3 SDA RC4 (I2C1): display on, 1030 data bytes`, followed, if the display is on, by the picture drawn as text. If the display did not answer, `NOT WIRED: <reason>` is appended.

Notes (standard error). `note: …` lines for things the model flags, and finally `note: unmodelled SFRs touched: …`, the registers of unmodelled peripherals that the program used. Notes go to standard error, so the report on standard output stays clean for scripts.

Examples

Run a program to its halt and print two variables:

```
basil-sim dist/test.hex --device 18F16Q41 --dump T_R1:2,T_R2
```

Run with serial input and check the transmit text:

```
basil-sim serial_rx.hex --device 18F27Q10 --stim serial_rx.stim.toml --dump SRX_B0,SRX_FERR
```

Blink program for one second of simulated time, with LEDs described in the stimulus file (`--until` is given a symbol that does not occur, so the run is limited by `--max-cycles`: 16 000 000 cycles is one second at 64 MHz):

```
basil-sim tests/sim/blink_q10.hex --device 18F27Q10 --stim tests/sim/blink.stim.toml --max-
cycles 16000000
```

Measure cycles between two routines:

```
basil-sim prog.hex --device 18F16Q41 --bp T_START,T_END --bp-max 2
```

The two `bp` lines give the cycle counts; subtract them. Divide by `Fosc / 4` for time.

Run paced at 1/10 of real time on a 32 MHz clock (the report is still printed only at the end; for live feedback use `pic-sim`):

```
basil-sim prog.hex --speed 1/10 --fosc 32 --max-cycles 3200000
```

Print the instructions between cycles 1000 and 1040:

```
basil-sim prog.hex --trace 1000-1040
```

On a PIC18 each trace line shows the cycle, address, mnemonic, the cycles the instruction took (`+n`) and the values of `TMR0L`, `PIR0` and `INTCON`; on a PIC16 it shows `W` and `STATUS`.

7. The stimulus file

A stimulus file is a TOML file that describes everything the outside world does to the part. It is given with `--stim FILE` or the *Stim…* field. Each section is written as `[[name]]` and may be repeated. **Time** is always an instruction cycle count from reset, written `at`; an entry with `at = 0` (the default) applies from the start. Where several entries set the same thing, **the later one replaces the earlier from its `at` on**.

An unknown section or key is an error, not ignored, so a misspelled name cannot silently do nothing. The largest value of `at`, `bit` and `gap` is 2^32.

A stimulus file is replayed from the start whenever the part is reset.

Pins: `[[pin]]` and `[[port]]`

Drive the level of input pins.

```
[[pin]]            # one pin
port = "B"         # letter A..H
bit = 5            # 0..7
level = 1          # 0 or 1 (or true/false)
at = 0

[[port]]           # several pins at once
port = "B"
value = 0x21       # levels of the pins in mask
mask = 0xFF        # default 0xFF
at = 20000
```

An input pin that nobody drives reads **0**. A pin shows the driven level only when it is an input (`TRISx` bit set) **and** digital (`ANSELx` bit clear); an output pin shows its latch.

Serial input: `[[uart]]`

Characters sent to the receiver of an EUSART (PIC18-Q10, all PIC16) or UART (PIC18-Q41).

```
[[uart]]
port = 1             # EUSART/UART number, default 1
at = 20000           # cycle at which the first Start bit falls, default 0
bytes = "Hello\r"    # a string, or:  data = [0x48, 0x165]
bit = 139            # the sender's bit time in instruction cycles; default: the receiver's
        own rate
gap = 0              # idle cycles between characters, default 0
nine = false         # 11-bit characters: data values up to 0x1FF, bit 8 is the ninth bit
bad_stop = false     # send the Stop bit as 0 (a framing error)
```

Give exactly one of `bytes` and `data`. A character whose Start bit falls while the receiver is off is lost, as on the part. The receiver's FIFO, framing and overrun errors behave as the data sheet describes (`docs/FACTS-SIM.md`).

ADC input: `[[adc]]`

The conversion result for a channel.

```
[[adc]]
channel = 16     # the value written to ADPCH (or the channel number of the simple ADC)
value = 0xABC    # the code the conversion produces, 0 .. 2^bits - 1
at = 600
```

The simulator does not model voltages here; it returns the code you give (limited to the converter's resolution and justified as the ADC's own settings require) for the channel selected when the acquisition ends. A channel with no entry, or one whose first entry is still in the future, converts as 0.

Analogue pin voltage: `[[analog]]`

The voltage on a pin, for the comparators (PIC18-Q10 only).

```
[[analog]]
pin = "RA0"
volts = 1.5      # 0 .. 6
at = 0
```

I2C client: `[[i2c]]`

A device on the I2C bus of a hardware host (PIC18 only: Q10 MSSP, Q41 I2C).

```
[[i2c]]
port = 1             # MSSP / I2C module number, default 1
address = 0x50       # 7-bit address (required)
data = [0xCA, 0xFE]  # bytes returned to the host's reads, in order
fill = 0xFF          # returned after data is used up, default 0xFF
```

The client acknowledges its address and every byte written to it, and answers reads from `data`. Several `[[i2c]]` entries make several clients; two at one address on one port are an error. A host that addresses an unlisted device gets a NACK (shown as `!` in the `i2c` report), and a read from nobody returns `0xFF`. The bus is assumed to have pull-ups.

SPI client: `[[spi]]`

The device at the other end of an SPI host (PIC18 only).

```
[[spi]]
port = 1               # SPI / MSSP number, default 1
data = [0xEF, 0x40]    # the byte returned for each byte the host exchanges, in order
fill = 0               # after data is used up, default 0
```

Reset pin: `[[mclr]]`

Hold the part in reset from a given time (PIC18 only).

```
[[mclr]]
level = 0        # 0 holds the part in reset when the pin is configured as MCLR
at = 20000
[[mclr]]
level = 1
at = 40000
```

The pin is high until driven.

Clock for the display: `fosc_mhz`

```
fosc_mhz = 64     # the PIC clock, 0 < MHz ≤ 64
```

An optional top-level key. It gives the clock used for the **OLED's scrolling** time (the OLED's own timing is in real time, the program's in cycles). Default 64.

Board parts: `[[led]]` and `[[oled]]`

See section 8.

What each family accepts

Section	PIC18 Q10	PIC18 Q41	PIC16
`[[pin]]`, `[[port]]`	yes	yes	yes
`[[uart]]`	yes	yes	yes
`[[adc]]`	yes	yes	yes
`[[led]]`, `[[oled]]`	yes	yes	yes
`[[i2c]]`, `[[spi]]`	yes	yes	**no**
`[[analog]]` (comparators)	yes	—	**no**
`[[mclr]]`	yes	yes	**no**

On a PIC16 part a section marked *no* is an **error** (`[[spi]] is not modelled on PIC16 parts yet…`) rather than being ignored. A PIC16 stimulus is also an error if it names a pin or port the part lacks, an EUSART it does not have, or an OLED on an I2C module the simulator does not model for that part (use `bus = "gpio"` instead).

Examples from the repository

`tests/sim/` holds working stimulus files with the BASIC programs they go with: `serial_rx.stim.toml` (three good characters then one with a bad Stop bit), `adc_q41.stim.toml` (a channel stepping from

0xABC to 0x123 at cycle 600), `i2c_q10.stim.toml`, `spi_q10.stim.toml`, `spi_q41.stim.toml`, `blink.stim.toml` and `board_example.stim.toml`. `tests/sim/run.sh` runs them (it needs a compiled `basil`).

8. The simulated board: LEDs and OLED

The simulator can attach two kinds of part to pins: LEDs and an SSD1306 OLED display. Describe them in the stimulus file, or in the **Board** tab of `pic-sim`.

LEDs

```
[[led]]
pin = "RB0"        # the pin the LED is wired to
active = "high"    # "high" (default): lit when the pin is a high output
name = "heartbeat" # optional
```

`active = "low"` is an LED wired to Vdd through the pin: lit when the pin is a **low** output. An LED is lit only while its pin is an **output** (the TRIS bit is clear) at the active level; an input pin leaves it dark. The window shows brightness from the fraction of time lit; `basil-sim` prints the final state, the time lit and each change with its cycle.

In the Board tab each LED has a name, a pin chosen from the part's pins, an *active high* check box and an `x` to remove it; **Add LED** adds another.

SSD1306 OLED

```
[[oled]]
bus = "i2c1"          # "i2c1"/"i2c2": the part's hardware I2C host; "gpio": bit-banged
scl = "RC3"
sda = "RC4"
address = 0x3C        # 7-bit, default 0x3C
height = 64           # 64 (default) or 32
external_vcc = false  # true: the module has its own VCC and lights without the charge pump
```

The display is 128 columns by 64 or 32 rows. **The pins you give must be the pins the program uses.**

Hardware bus (`i2c1`, `i2c2`). The OLED acknowledges only when the program has set up the module's pin selection as it would have to on a real part: the input selectors (`SSPxCLKPPS`, `SSPxDATPPS` on the Q10 and PIC16; `I2CxSCLPPS`, `I2CxSDAPPS` on the Q41) must select the OLED's SCL and SDA pins, and the output selector `RxyPPS` of **both** pins must hold the bus output code (Q10 MSSP1: SCL `0x0F`, SDA `0x10`; MSSP2: `0x11`, `0x12`; Q41 I2C1: SCL `0x21`, SDA `0x22`). Otherwise the address gets a NACK, the display does not draw, and the Board tab or `basil-sim` says which register is wrong (`NOT WIRED: …`). A PIC16 part must have the MSSP in I2C host mode for this.

Bit-banged (`gpio`). The two pins are watched after every instruction. A start or stop is a change of SDA while SCL is high, a bit is SDA at the rising edge of SCL, and the display pulls SDA low for the acknowledge. A line is low when the PIC drives it low or the display holds it; external pull-ups are assumed, so a released line reads high.

What the display understands. The simulator implements the SSD1306 command set from its data sheet: display on/off, contrast, the three addressing modes with the data sheet's pointer rules, column and page range, start line, display offset, multiplex ratio, segment remap, COM scan

direction, invert, entire display on, the charge pump, and the horizontal and the vertical-and-horizontal **scroll** commands. Scrolling runs in real time from the `fosc_mhz` clock; at reset one scroll step of five frames is about 47 ms.

The charge pump. The panel lights only when the charge pump has been enabled (`8Dh 14h` before `AFh`), or when `external_vcc = true`. A program that turns the display on without either gets a note and a dark panel; the Board tab shows `on, but dark: charge pump (8Dh 14h) not enabled` and an *external VCC* check box.

Reads. I2C reads from the display return `0xFF` and a note, because the SSD1306 has no read path in serial mode.

In the window. The Board tab has a bus choice (hardware I2C1, hardware I2C2 or bit-banged GPIO), SCL and SDA pin lists, the address (hex), 64 or 32 rows, and the external-VCC option. **Add OLED** is offered when none is wired. Changing a pin or address re-attaches the part: the display starts blank like a power-up. The Board tab can be used before a program is loaded and the wiring is kept when another program is loaded.

9. What is simulated

Time

- The unit is the instruction cycle (`Fosc / 4`). Instructions take the cycle counts in the data sheet's instruction tables: one cycle for most; two for branches, calls, returns, table reads and writes and `MOVFF` on PIC18; extra cycles for a taken skip; one extra for a PIC16 access to program flash through an indirect register.
- Peripheral events are scheduled at exact cycles. A timer flag, a UART bit time, an ADC conversion and an EEPROM write complete at the cycle the data sheet gives.
- Interrupt entry follows the data sheet (and was matched against MDB): the vector is reached a fixed number of cycles after the flag is set and after the instruction in progress completes.

Cores

	PIC18	PIC16 enhanced mid-range
Instruction set	Base set plus the Q41 additions (`MOVFFL` …)	The 50-instruction set including `MOVIW`, `MOVWI`, `BRW`, `CALLW`, `ADDFSR`
Program counter	Bytes, 21 bits	Words, 15 bits
Stack	Hardware, 31 levels (Q10), overflow/underflow reset with `STVREN`	16 levels; the 17th push overflows; `STKPTR` as the data sheet
Interrupts	Q10: legacy, optional two priorities, vectors 0008h and 0018h. Q41: with `MVECEN` = 0, hardware context save	Single vector 0004h, automatic save of W, STATUS, BSR, FSRs and PCLATH to the shadow registers
Data memory	Banked with Access Bank, FSR0–2	Banked (6-bit or 5-bit BSR), linear memory through the FSRs, flash read through an FSR

Peripherals

Models exist for the following; a register of anything else is plain storage that honours the data sheet's implemented-bit and read-only masks and is reported as "unmodelled SFRs touched" when the program uses it.

Peripheral	PIC18-Q10	PIC18-Q41	PIC16
Ports, `PORTx/LATx/TRISx/ANSELx`, PPS and `PPSLOCK`	yes	yes	yes
Interrupts, `SLEEP` and wake-up	yes	yes	yes
Timer0, Timer1, Timer2 (Timer3–6 where present)	yes	yes	yes
EUSART / UART	EUSART1/2, asynchronous	UART1–3	EUSART, asynchronous
ADC	yes	yes (12-bit)	yes (several variants)

Peripheral	PIC18-Q10	PIC18-Q41	PIC16
Data EEPROM	yes	yes	yes
Program flash self-write	yes	yes	yes
MSSP (SPI and I2C host)	yes	—	I2C host mode, to drive the OLED
SPI host	via MSSP	yes	—
I2C host	via MSSP	yes	—
CCP1/2 and PWM3/4	yes	—	—
CWG1	yes	—	—
Comparators C1/C2	yes	—	—
CLC1–CLC8	yes	—	—
Windowed watchdog and the non-power-on resets	yes	—	`CLRWDT` only
CRC and memory scanner	yes	—	—
External interrupts INT0–2, interrupt-on-change	yes	—	—
MCLR pin reset	yes	yes	the MCLR pin is part of the port model
LEDs and SSD1306 OLED	yes	yes	yes

Where this table says "—", `docs/FACTS-SIM.md` documents no model for that peripheral on that family: its registers are plain storage and the program is reported as touching unmodelled SFRs. (The watchdog, CRC, comparator, CLC, CWG, CCP and external-interrupt models are written and cited against the Q10 data sheet.) `docs/FACTS-SIM.md` is the authoritative list, section by section, with the data sheet page for each rule; peripherals are added over time.

Not modelled anywhere: DMA, the Q41 vectored-interrupt table (`MVECEN = 1`; a note appears and the interrupt goes to the legacy vector), synchronous EUSART mode, weak pull-ups, open-drain outputs, input thresholds, the oscillator start-up and clock switching (the oscillator registers take their reset values from the configuration words, but `CSWEN` and the ready bits are not applied), brown-out, `LVP`, code and write protection. The watchdog is documented for the PIC18-Q10; on a PIC16 `CLRWDT` only sets TO and PD and the watchdog never times out.

Configuration words

`RSTOSC` sets the reset values of `OSCCON1`, `OSCCON2` and `OSCFRQ`; `WDTCPS`, `WDTCWS` and `WDTCCS` set the watchdog registers' reset values; on a PIC16 `MCLRE`, `PPS1WAY` and `STVREN` take effect, and `STVREN` also decides whether a PIC18 stack overflow resets the part. `LVP`, code protection and write protection have no effect. `basil-sim --config` (PIC18 only) prints every field and the registers derived from them.

Deliberate differences from MDB

The simulator follows the data sheet. Where it differs from MDB it says so in `docs/FACTS-SIM.md`:

- **D-SIM-1:** MDB loses part of the Timer0 prescaler count on a read of `TMR0L`; the data sheet clears the prescaler on writes only.
- **D-SIM-2 (open):** interrupt entry racing a `bcf INTCON,7` critical section differs from MDB by a few cycles.
- **D-SIM-5:** on PIC16F183xx with `PR2 = 0` MDB never sets the Timer2 flag; the simulator does, every prescaler × postscaler cycles, as the data sheet says.
- MDB has no write time for EEPROM and flash; the simulator does, so cycle counts differ there while results agree.

10. Troubleshooting

Symptom	Cause and remedy
The device box is empty	`devices/` was not found at its build-time location. Rebuild from the workspace that holds it (section 2)
`symbol __halt not found (no prog.S)`	`basil-sim` found neither a listing nor an ELF with `__halt`. Build with `-g`, give `--asm`, or name another symbol with `--until`
`basil-sim` runs for a long time and exits with status 1	The program never reaches `__halt` (a main loop). Add `--max-cycles` for the time you want to simulate
`unknown option, needs a value`	Check the option spelling; `--dump`, `--bp` and `--until` take a value
`no .elf beside the hex: no source view or variables`	Build with `basil -g` and keep the `.elf` beside the `.hex` (or one folder up from a `dist/` hex)
`ELF has no BASIC lines`	The ELF came from a build without `-g`, or from a part for which basil emits no line information. Disassembly view, SFRs and Watch still work
`<part>: …` error when loading a stimulus	A section or key the part cannot honour, or a pin the part lacks (section 7). Read the message; it names the section
The LED never lights	The pin is an input (TRIS set), is on the wrong port, or `active` is wrong. In the program, set `TRISx` and check `ANSELx`
The OLED shows `NOT WIRED`	For a hardware bus the program's PPS set-up does not match the pins you gave: the message names the register. Check `SSPxCLKPPS` / `SSPxDATPPS` (or `I2CxSCLPPS` / `I2CxSDAPPS`) and both `RxyPPS` registers
The OLED is on but dark	The charge pump was not enabled (`8Dh 14h`) and `external_vcc` is false
Serial input is garbled or lost	Your `bit` or the program's baud rate differs; with `bit` omitted the sender uses the receiver's own rate. The receiver must be enabled (`SPEN`, `CREN`) before the Start bit falls
An input pin always reads 0	Nothing drives it in the stimulus (`[[pin]]` / `[[port]]`), or `ANSELx` has the bit set
A flag never sets	The peripheral may be unmodelled; look for `unmodelled SFRs touched` in the notes
Results differ from the real chip	See the notes and section 9. If a behaviour matters and is marked `[A#]` in `docs/FACTS-SIM.md`, it is an assumption not yet measured on silicon

11. Appendix A: quick reference

pic-sim controls

Where	Control	Effect
Toolbar	Run / Pause	Start, stop a run or step
Toolbar	Step / Step over	One instruction / run over a call
Toolbar	Line / Line over / Out	Source-line stepping (needs `basil -g`)
Toolbar	Reset	Restart the part, replay the stimulus, keep the wiring
Toolbar	Load	(Re)load the program and stimulus; clears breakpoints
Toolbar	Stim… / x	Choose a stimulus file (opens in its folder, the program's, or `tests/sim`) / clear it
Toolbar	Speed, Fosc MHz	Run speed and display clock
Program pane	click a line	Toggle a breakpoint
Program pane	follow PC, stay in my code, goto/Go	View control
CPU panel	Stopwatch Start/Stop/Lap/Clear	Time simulated intervals
Right tabs	Memory, SFRs, Watch, Board, Variables	Inspection and wiring
Window	drop `.hex` / `.elf` / `.toml`	Load the program / set the stimulus

basil-sim options

`--device`, `--devices`, `--asm`, `--until`, `--max-cycles`, `--stim`, `--dump`, `--bp`, `--bp-max`, `--trace`, `--config`, `--speed`, `--fosc`, `--version`.

Stimulus sections

`[[pin]]`, `[[port]]`, `[[uart]]`, `[[adc]]`, `[[analog]]`, `[[i2c]]`, `[[spi]]`, `[[mclr]]`, `[[led]]`, `[[oled]]`, and the top-level `fosc_mhz`.

Pin names

`R<port><bit>` with port `A` to `H` and bit `0` to `7`: `RA0`, `RC3`, `RB7`. Only the pins the selected part has are valid.

Run speeds

`Max`, `1x (real time)`, `1/2`, `1/4`, `1/10`, `1/100`, `1/1000` in the window; `max`, or any ratio such as `1`, `0.5`, `1/10` on the command line.

Exit status of basil-sim

0 reached `--until`; 1 stopped for another reason; 2 error.

12. Appendix B: example session

The repository's `crates/picsim/tests/data/zed_g.hex` is a clock/calendar program for an external I2C real-time clock chip, built with `basil -g`. Running it on a simulated PIC18F27Q10 with nothing attached to the I2C bus:

```
$ basil-sim crates/picsim/tests/data/zed_g.hex --device 18F27Q10 --max-cycles 5000000
stop: Cycles  cycles: 5000001  instructions: 3334050
tx1: \r\nMCP7940N clock/calendar\r\nRTCC not answering, status 3\r\n
i2c1: S 6FW! P S 6FW! P S 6FW! P S 6FW! P
note: unmodelled SFRs touched: RX1PPS SP1BRGH SP1BRGL SSP1CLKPPS SSP1DATPPS
```

Reading it:

- The program did not halt in the 5 000 000 cycles allowed (`stop: Cycles`), which is 0.31 s at 64 MHz, and `basil-sim` would exit with status 1.
- It printed its banner on the serial port and then reported that the real-time clock chip did not answer (`tx1`).
- The `i2c1` line shows why: four times the host started a transfer to address 6F for writing and the byte was not acknowledged (`!`), followed by a stop. No device listens at that address.
- Four SFRs are listed as unmodelled for this part; they belong to peripherals whose models do not cover those registers.

To make the chip answer, give the I2C bus a client in a stimulus file:

```toml
# rtcc.stim.toml
[[i2c]]
address = 0x6F
data = [0x00, 0x00, 0x00]
```

and run again with `--stim rtcc.stim.toml`. The `!` marks disappear from the `i2c1` line and the program carries on to set the clock.

To look at the same program in the window:

```
pic-sim crates/picsim/tests/data/zed_g.hex
```

The device is taken from the ELF, the Source pane opens at the program's `.bas` file, and the Variables tab shows its globals. Click the line that sends the banner, press **Run**, and the program stops there; press **Line over** to step to the next line and watch the orange variables change.